SSH连接

一、什么是SSH连接

SSH是一种加密网络协议，用于在不安全的网络环境中实现安全的远程系统访问和数据传输。SSH连接则是指通过SSH协议在客户端和服务器之间建立的通信通道。

二、SSH连接建立详解

1.连接建立过程

- **建立基础连接：**客户端向服务器22端口（默认）发起TCP连接请求，完成三次握手建立基础通信通道。服务器sshd守护进程持续监听22端口readiness。
- **协商协议版本：**
  - 版本声明：客户端发送 `SSH-2.0-<客户端版本>` 标识（如 `SSH-2.0-OpenSSH_8.6`），服务器回应 `SSH-2.0-<服务端版本>`（如 `SSH-2.0-OpenSSH_9.4p1`）
  - 版本匹配：双方选择共同支持的最高协议版本（通常为SSH2.0），若不兼容则断开连接。
- **算法协商：**
  - 双方声明所支持算法列表，包括：
    - 密钥交换算法（如curve25519-sha256）
    - 加密算法（如aes256-gcm@openssh.com）
    - MAC算法（如hmac-sha2-512）
    - 压缩算法（如zlib@openssh.com）
  - 算法选择：按优先级依次匹配双方均支持的算法，优先选择高安全性算法。
- **密钥交换阶段：**服务器和客户端通过密钥交换算法，动态生成共享的会话密钥和会话ID，建立加密通道。会话密钥主要用于后续数据传输的加密，会话ID用于在认证过程中标识该SSH连接。
  - Diffie-Hellman密钥交换：
    - 服务器生成随机数b（用于生成本次通讯的私钥）和素数P
    - 客户端生成随机数a，用于生成本次通讯的私钥
    - 双方分别用各自的私钥、共同的加密生成器、和共同的素数生成各自的公钥，并将公钥发送给对方
    - 双方通过各自的私钥和对方发过来的公钥生成另一个密钥。根据该算法，双方各自计算出来的两个密钥是完全一样的。

        - 这个共享密钥的加密方式被称为二进制数据包协议。该过程能够让双方平等的参与密钥生成的过程，而不是由单方掌握。这种共享密钥生成的过程是安全的，双方没有交换过任何未经加密的信息。
    - 最终生成的密钥是对称式密钥，一方用于加密信息的密钥等同于另一方用于解密信息的密钥，而任何第三方由于不持有该密钥，是无法解密双方传递的内容的。
- **服务器认证用户开放权限：**服务器验证用户身份以决定是否准许其访问。验证有不同的方式，选择的验证方式取决于服务器的支持。SSH认证有两种方式：
    a. 密码验证。服务器要求客户端输入密码，客户端输入的密码经过上述的通道加密传输给服务器。虽然密码是加密过的，然而该方法仍然不被推荐，因为用户经常为了省事而使用过于简单的密码，而这类密码很容易就能够被自动化脚本破解。
    b. SSH秘钥验证。最流行的验证方式是SSH密钥对，这也是当前最推荐的方式。

    SSH密钥对 是非对称密钥，私钥和公钥分别用于不同的功能。公钥用于加密，而私钥用于解密。公钥可以随意上传、共享，因为公钥的流通并不会危及到私钥的保密性。SSH密钥对的验证过程起始于上一部分加密通道建立之后，其具体执行步骤如下：

    a. 客户端发送自己的密钥ID给服务器端
    b. 服务器在自己的 authorized_keys 文件中检查是否有此ID的公钥
    c. 如果有，则服务器生成一个随机数，用该公钥加密之
    d. 服务器将加密后的随机数发给客户端
    e. 客户端用私钥解密该随机数，然后在本地为随机数做MD5哈希
    f. 客户端将该MD5哈希发给服务器端
    g. 服务器端为一开始自己生成的随机数也做一个MD5哈希，然后用通讯通道“公共的密钥”将该哈希加密，再跟客户端发来的内容进行对比。如果双方内容一致，则通过验证，开放访问权限给客户端

身份验证通过，建立加密会话通道，此后通过密钥交换阶段生成的公钥对传输的信息进行对称加密。

2.总结

SSH连接过程中一共使用了两种密钥，一个是在本地生成的自己的公钥私钥，公钥提交给服务器，用于用户身份验证，一个是交换通过随机数生成的对称密钥，用于会话数据传输。

三、SSH公私钥的生成

我们在上面的连接介绍中，提到重要的一环，即把自己的公钥发送给服务器。那么，我们怎么创建自己的公私钥呢。

1.基础生成命令

```
代码块
ssh-keygen -t ed25519 -C "your_email@example.com"
```

- 参数说明：
  - `-t`：指定加密算法类型
    - `ed25519`：推荐，现代高效算法（比 RSA 更安全且密钥更短）
    - `rsa -b 4096`：传统算法，需指定位数（兼容性更好但安全性较低）
  - `-C`：注释信息（建议用邮箱或设备标识）
  - `-f ~/.ssh/custom_key_name`：自定义密钥文件名
  - `-N "your_passphrase"`：为私钥设置密码（可选）

一般不需要给自己的私钥设置密码，不同服务的密钥最好放在不同的文件夹下，用-f为自己的密钥选择对应的存储位置。不同的存储位置，和不同的注释，标志着和不同的服务器建立的连接。

2.生成过程详解

```
代码块
$ ssh-keygen -t ed25519 -C "dev@workstation"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/user/.ssh/id_ed25519):  # 建议保留默认路径
Enter passphrase (empty for no passphrase):  # 可留空或设置密码
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_ed25519
Your public key has been saved in /home/user/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:XXXX dev@workstation
```

路径可以在命令处指定，也可以在此处，若是在命令处指定，此处可以直接回车，密码处同理，不需要密码可以直接回车。

命令执行完毕，生成两个文件：

- 私钥：`~/.ssh/id_ed25519`（需严格保护权限），权限不能太过宽松，会导致系统认为它不安全，从而使用失败。
- 公钥：`~/.ssh/id_ed25519.pub`（可公开）

四、SSH Config

SSH Config 是 OpenSSH SSH 服务器守护进程的配置文件，用于设置 SSH 服务的相关参数，如监听地址、端口、认证次数限制、是否允许 root 登录等。

1.配置文件位置

- 用户级配置：`~/.ssh/config`（Linux/Unix/Mac）,C:\Users\Lenovo\.ssh\config（Windows）
- 系统级配置：`/etc/ssh/ssh_config`
- 优先级：命令行参数 > 用户配置文件 > 系统配置文件

2.文件权限

`~/.ssh/config` 设置为 `600`（Linux中带代表仅所有者可读/写），

3.文件结构

代码块

```
Host <别名>
    参数1 值1
    参数2 值2
Host <另一个别名>
    参数A 值A
```

主要的规则如下：

1. 每项配置都是 `参数名 参数值` 或 `参数名=参数` 值的形式，其中参数名不区分大小写，而参数值区分大小写，如上面的参数名 `HostName` 和 `Hostname` 是同一个参数
2. 不同主机的配置通过 `Host` 参数来区分，一个配置文件里面可以有针对多个Host的配置
3. 以 `#` 开头的是注释，会被忽略
4. 同一个Host的配置内部， `参数名 参数值` 和 `参数值=参数名` 的形式可以混用

常见参数：

Host

类似昵称，用于标识某个特定的配置，在ssh命令中使用，如在命令行执行如下命令即可：

代码块

```
ssh cluster
```

一个最有用的场景是使用scp在不同主机间传数据。没有配置之间，你得写很长的参数，如

代码块

```
scp a.txt tom@192.168.11.11:~/
```

尤其是IP地址记忆起来好麻烦啊。配置过上例中的文件后，这个任务可以简化成这样：

代码块

```
scp a.txt cluster:~/
```

省略了用户名和IP地址，方便多了。

HostName

HostName 是 SSH 配置文件（~/.ssh/config）中用于指定实际连接目标主机名或 IP 地址的指令。它的核心作用是将 SSH 命令中的别名（Host）映射到真实的主机地址。

User

登录主机的用户名

IdentityFile

认证证书文件，默认位置是 `~/.ssh/id_rsa`，`~/ssh/id_dsa` 等，如果采用默认的证书，可以不用设置此参数，除非你的证书放在某个自定义的目录，那么你就需要设置该参数来指向你的证书。

Port

SSH访问主机的端口号，默认是22端口，同上，只有在非默认情况下才需要设置该值

示例：

代码块

```
Host my_server
    HostName 192.168.1.100
    User admin
    PreferredAuthentications publickey
    IdentityFile ~/.ssh/id_rsa
    IdentitiesOnly yes  # 仅使用指定密钥
    Port 2222
Host server_1
    HostName 10.0.0.1
Host server_2
    HostName 10.0.0.2
```

可以在其中写多个服务，对应不同的服务器。

四、github配置（SSH使用示例）

我们了解到，SSH用来进行远程系统访问和数据传输，那么我们在本地通过git向github推送和拉取代码，本质上也是一种远程系统访问和数据传输。所以，在这里通过对github的SSH配置，来捋一下SSH连接建立的过程。

首先，我们在自己电脑上生成对应的公钥私钥，使用命令：

代码块

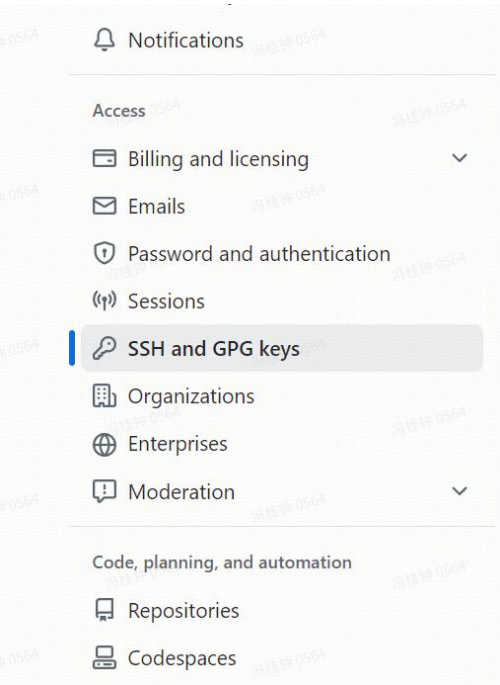
```
ssh-keygen -t rsa -C "xxx@xxx.com"#最好是自己的邮箱名
```

然后，获取SSH公钥的内容，通过命令

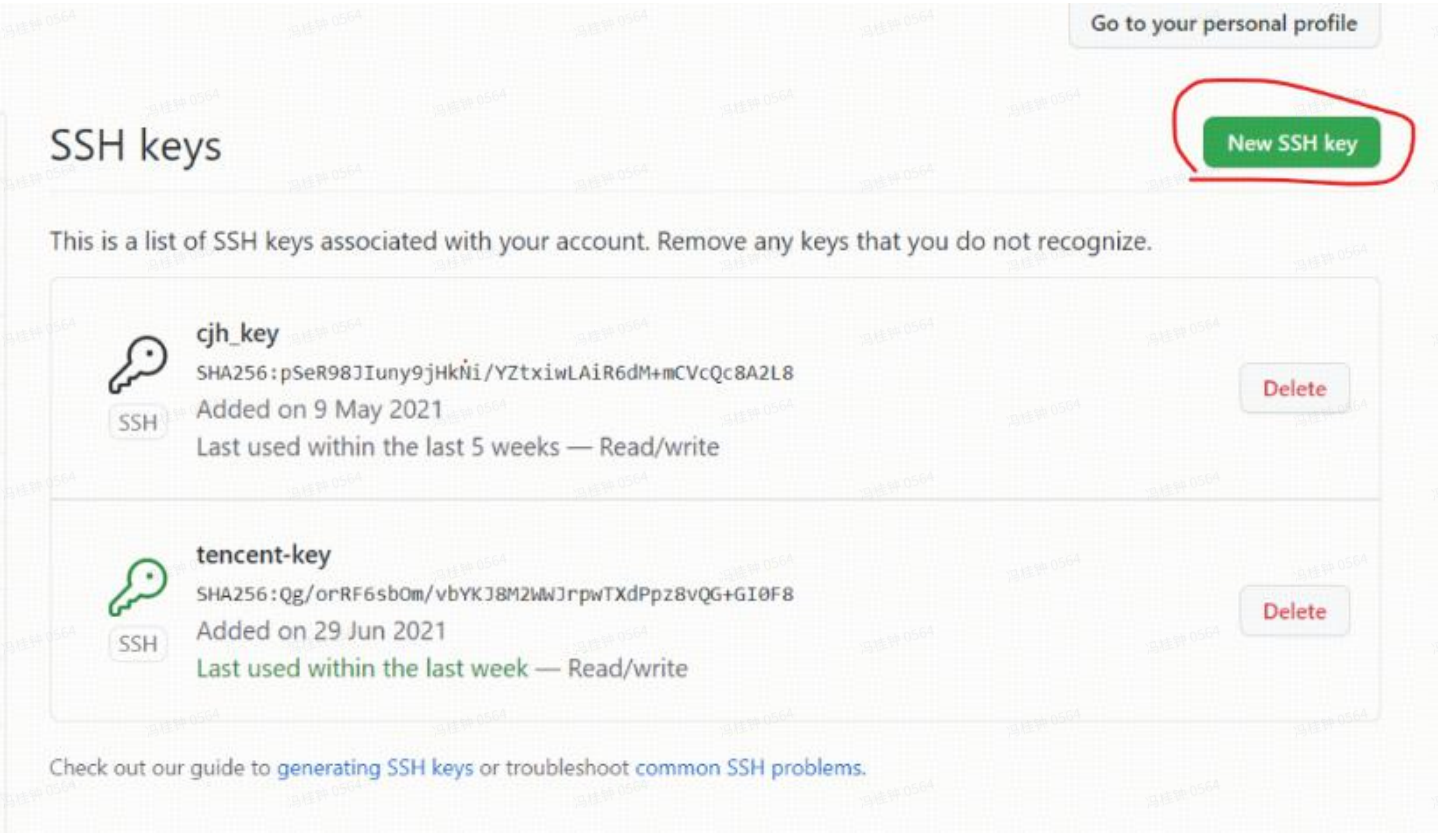
代码块

```
cd ~/.ssh
cat id_rsa.pub（此处应该是你存储github密钥得文件）
```

接着，进入github，找到设置，进入添加密钥的地方



复制粘贴自己的公钥



最后，通过命令验证是否实则成功

代码块

```
ssh -T git@github.com
```

```
$ ssh -T git@github.com
Hi jiahao12121! You've successfully authenticated, but GitHub does not provide s
hell access.
```

如图则配置成功

五、总结

通过上面的配置流程，我们来回顾一下SSH建立的过程：

SSH登录安全性由非对称加密保证，产生密钥时，一次产生两个密钥，一个公钥，一个私钥，在git中一般命名为id_rsa.pub, id_rsa。

本地生成一个密钥对，其中公钥放到远程主机，私钥保存在本地

当本地主机需要登录远程主机时，本地主机向远程主机发送一个登录请求，远程收到消息后，随机生成一个字符串并用公钥加密，发回给本地。本地拿到该字符串，用存放在本地的私钥进行解密，再次发送到远程，远程比对该解密后的字符串与源字符串是否等同，如果等同则认证成功。

对于对称加密和非对称加密的理解，可以参考文章：

https://javaguide.cn/cs-basics/network/http-vs-https.html#ssl-%E5%92%8C-tls-%E7%9A%84%E5%8C%BA%E5%88%AB